

Title: Non-Linear Out-of-Order Bias (NL-OOB) in Transformer Attention: A Specification and Theoretical Framework for Structural Generalization

1. Executive Summary

The Transformer architecture has fundamentally reshaped the landscape of artificial intelligence, establishing itself as the de facto standard for natural language processing, computer vision, and increasingly, scientific discovery. Its core mechanism, Self-Attention, offers a powerful inductive bias for modeling global dependencies by treating input tokens as a set. However, this permutation invariance necessitates the injection of structural information to model sequences, images, or graphs effectively. While initial approaches relied on Absolute Positional Encodings (APE) and later Relative Positional Encodings (RPE), these methods have consistently struggled with **length extrapolation**—the ability to generalize to contexts longer than those seen during training.

A pivotal advancement in this domain is **Attention with Linear Biases (ALiBi)**, which replaces learned embeddings with a static, non-learned penalty subtracted from attention scores based on the linear distance between tokens. By strictly enforcing a "recency bias" through geometric scaling of slopes across attention heads, ALiBi achieves state-of-the-art extrapolation capabilities in autoregressive language modeling.¹ However, a detailed inspection of the ALiBi design reveals inherent limitations: its reliance on linear decay ($|i-j|$) and sequential indexing renders it suboptimal for **non-sequential domains**—such as molecular geometry, knowledge graphs, and hierarchical code structures—where interactions are governed by non-linear physical laws or topological distances rather than sequence order.³

This report presents a comprehensive specification for **Attention with Non-Linear Out-of-Order Bias (NL-OOB)**, a novel architectural paradigm designed to generalize the extrapolation benefits of ALiBi to arbitrary metric spaces. Unlike ALiBi, which presumes a linear 1D sequence, NL-OOB decouples the bias calculation from input indices, employing a **permutation-equivariant distance tensor** derived from the intrinsic topology of the data (e.g., Euclidean distance in 3D space, geodesic distance in graphs). Furthermore, motivated by findings in neuroscience regarding power-law adaptation in visual cortexes⁵ and physical laws governing atomic interactions⁶, NL-OOB replaces the linear penalty with a **parametric non-linear decay function** (specifically logarithmic or power-law profiles).

The specification details a mechanism for **Geometric Head Scaling** that adapts ALiBi's slope

initialization logic to non-linear exponents, creating a multi-scale attention topology where different heads specialize in different interaction ranges—from covalent-bond-like locality to global conformational awareness. We provide a rigorous theoretical analysis comparing NL-OOB to existing methods like Graphormer and Kerple, demonstrating that NL-OOB offers a superior inductive bias for continuous extrapolation in structural domains.

Finally, the report illustrates the practical utility of NL-OOB through two detailed use cases: a **Molecular Geometry Transformer** for quantum property prediction, and a **Hierarchical Code Analyzer** for abstract syntax tree processing. These examples demonstrate how NL-OOB can effectively replace computationally expensive Message Passing Neural Networks (GNNs) with a more parallelizable, extrapolation-friendly attention mechanism.

2. The Evolution of Positional Inductive Biases

To understand the necessity and design of Non-Linear Out-of-Order Bias, one must first deconstruct the trajectory of positional encoding schemes. This evolution reflects a deepening understanding of how neural networks process "order" and "structure," moving from rigid, absolute coordinate systems to flexible, relative, and eventually topology-aware mechanisms.

2.1 The Limitations of Absolute and Learned Embeddings

The original Transformer introduced by Vaswani et al. relied on sinusoidal Absolute Positional Encodings (APE).⁷ This method assigns a unique, fixed vector to each position index i and adds it to the token embedding. While mathematically elegant—leveraging the property that $PE(i+k)$ is a linear function of $PE(i)$ —APE suffers from a critical flaw: it is not translation invariant. The model learns to attend to "position 5" rather than "5 tokens back." Consequently, when a model trained on 512 tokens encounters token 513, the absolute embedding is undefined or out-of-distribution, leading to catastrophic performance degradation.¹

Learned absolute embeddings, used in models like BERT and GPT-2, fare no better. They simply learn a lookup table for positions 0 to N_{max} . Extrapolation is mathematically impossible because the parameters for positions $> N_{\text{max}}$ do not exist.¹⁰ This limitation sparked the shift toward **Relative Positional Encodings (RPE)**, as seen in T5 and Transformer-XL.¹¹ RPE encodes the distance $i-j$ rather than positions i and j . While T5's bucketed bias improves generalization, it introduces additional learnable parameters and computational overhead, often slowing down inference.¹²

2.2 The Rotation Paradigm: RoPE

Rotary Positional Embeddings (RoPE) represented a significant leap by encoding position

directly into the rotation of the Query and Key vectors.¹² By rotating the vectors by an angle proportional to their position, the dot product $\mathbf{q}_i^T \mathbf{k}_j$ becomes a function of $i-j$. RoPE elegantly captures relative distance without auxiliary parameters. However, empirical studies suggest that while RoPE generalizes better than APE, it still struggles with significant length extrapolation (e.g., training on 2k tokens and testing on 8k) because the high-frequency components of the rotation become unstable or out-of-phase at unseen distances.⁹

2.3 The Paradigm Shift: ALiBi (Attention with Linear Biases)

Attention with Linear Biases (ALiBi) marks a fundamental divergence in design philosophy. Rather than adding information to the *content* (the embeddings), ALiBi modifies the *geometry* of the attention mechanism itself.

2.3.1 Detailed Inspection of ALiBi Design

Based on the implementation details provided in LabML¹⁵ and the original literature¹, ALiBi operates by subtracting a static, non-learned penalty from the attention scores before the softmax operation.

The attention score a_{ij} for head h is computed as:

$$a_{ij}^{(h)} = \text{softmax}(\mathbf{q}_i^T \mathbf{k}_j + b_{ij}^{(h)})$$
 where the bias term is:

$$b_{ij}^{(h)} = -m_h \cdot |i-j|$$

Here, $|i-j|$ represents the linear integer distance between the query index i and the key index j . The scalar m_h is a head-specific slope.

The Geometric Sequence of Slopes:

A crucial, often overlooked detail of ALiBi is how the slopes m_h are determined. They are not learned; they are fixed according to a geometric progression. For a model with H heads, the slopes are:

$$m_h = \frac{1}{2^{8 \cdot \frac{h}{H}}}$$

(Note: The exact formulation in the code 15 uses a base ratio derived such that the slopes range from $2^{-8/H}$ down to 2^{-8} .)

This geometric scaling is the "secret sauce" of ALiBi.¹³ It creates a multi-resolution attention mechanism:

- **High-slope heads ($m \approx 0.5$):** The penalty is severe. Attention decays rapidly as distance increases. These heads effectively act as **local convolution filters**, attending only to the immediate previous tokens. They are robust to extrapolation because they never "look" far enough to realize the sequence is longer than training length.

- **Low-slope heads ($m \approx 0.01$):** The penalty is gentle. These heads can attend to the distant past, capturing global context.

Limitations of ALiBi:

Despite its success in 1D language modeling, ALiBi's design contains implicit assumptions that fail in broader contexts:

1. **Linearity:** It assumes relevance decays exponentially with distance (since a linear penalty in log-logit space becomes exponential decay in probability space).
2. **Sequentiality:** It assumes distance is defined by integer indices i and j .
3. **1D Topology:** It cannot natively handle trees, graphs, or 3D structures without flattening them, which destroys topological information.

The **Non-Linear Out-of-Order Bias (NL-OOB)** proposed in this report addresses these limitations by generalizing the core insights of ALiBi—static, distance-based, geometrically scaled penalties—to non-linear functions and arbitrary metric spaces.

3. Theoretical Framework: Physics, Kernels, and Neuroscience

To justify the design of NL-OOB, we must look beyond engineering heuristics to the fundamental principles governing interaction decay in natural systems. The linear bias of ALiBi ($|i-j|$) is a specific instance of a broader class of kernels.

3.1 Kernel Methods and Conditionally Positive Definite Functions

The attention mechanism acts as a kernel smoother. The bias term B_{ij} effectively modifies the kernel. Research into **Kernelized Relative Positional Embeddings (Kerple)**¹⁶ establishes that for a positional bias to be valid, it should relate to **Conditionally Positive Definite (CPD)** kernels.

- The linear bias $|i-j|$ corresponds to the **Laplacian Kernel** ($e^{-|x|}$).
- A squared bias $|i-j|^2$ corresponds to the **Gaussian (RBF) Kernel** (e^{-x^2}).
- A logarithmic bias $-\log(1+|i-j|)$ corresponds to the **Rational Quadratic Kernel** or **Power Law Kernel**.

While ALiBi uses the Laplacian kernel, this is not necessarily optimal for all data types. Gaussian kernels are standard in computer vision for modeling smooth spatial proximity¹⁸, while Power Law kernels are increasingly recognized as superior for modeling hierarchical and scale-free networks, such as social graphs or semantic dependencies in language.³

3.2 The Physics of Attention: Power Laws

In physical systems, interactions rarely decay exponentially (linearly in log-space).

- **Gravity and Electrostatics:** Decay as $1/r$ (potential) or $1/r^2$ (force).
- **Dipole-Dipole interactions:** Decay as $1/r^3$.
- **Van der Waals forces:** Decay as $1/r^6$.

If we aim to build "AI Scientists" or models that understand the physical world (e.g., molecular transformers), the inductive bias of the attention mechanism should mirror these physical realities.⁴ A linear bias in the logit space implies $\text{Attention} \propto e^{-r}$, which vanishes far too quickly compared to $1/r^2$.

By using a logarithmic bias:

$$b_{ij} = -p \cdot \log(r_{ij}) \quad \text{The attention weight becomes: } \exp(\mathbf{q} \mathbf{k}^T - p \log r) = \exp(\mathbf{q} \mathbf{k}^T) \cdot r^{-p}$$

This Power Law Bias allows the model to learn interactions that decay slowly, maintaining significant attention mass even at long distances, preventing the "over-squashing" problem prevalent in GNNs and standard Transformers.¹⁹

3.3 Neuroscience and Efficient Coding

Evidence from neuroscience further supports non-linear biases. In the mouse primary visual cortex (V1), population responses adapt to stimulus statistics following a power law, a phenomenon linked to **efficient coding** principles.⁵ The brain allocates resources (attention/neurons) such that the response magnitude scales with the probability of the stimulus, typically governed by heavy-tailed distributions. This suggests that artificial attention mechanisms designed to process natural signals (images, audio) should also employ power-law scaling to maximize information transmission efficiency.

3.4 Out-of-Order Indexing and Permutation Equivariance

Standard Transformers are sequence processors. However, a "set of atoms" or a "database of facts" has no inherent sequence.

Permutation Equivariance is the property that:

$$f(\mathbf{P} \mathbf{X}) = \mathbf{P} f(\mathbf{X})$$

for any permutation matrix $\mathbf{P}$.

Standard ALiBi violates this because the bias $|i-j|$ depends on the index. If we swap the first and last token, the semantic meaning might not change (in a set), but the ALiBi bias changes dramatically.

NL-OOB ensures permutation equivariance by deriving the bias from a Distance Tensor $\mathbf{D}$ that permutes consistently with the input. If inputs are permuted $\mathbf{X}$

$\mathbf{P}\mathbf{X}$, the distance matrix transforms as $\mathbf{D} \rightarrow \mathbf{P}\mathbf{D}\mathbf{P}^T$, ensuring the bias between specific entities remains constant regardless of their input index.

4. Specification: Attention with Non-Linear Out-of-Order Bias (NL-OOB)

Based on the detailed inspection of ALiBi¹⁵ and the theoretical needs identified above, we define the specification for **Attention with Non-Linear Out-of-Order Bias (NL-OOB)**. This module serves as a drop-in replacement for standard Multi-Head Attention (MHA).

4.1 Core Mathematical Formulation

Let $\mathbf{X} \in \mathbb{R}^{B \times N \times D_{\text{model}}}$ be the input tensor. Let $\mathbf{D} \in \mathbb{R}^{B \times N \times N}$ be the Generalized Metric Tensor representing the pairwise distances between all entities in the batch. The attention scores $\mathbf{A}^{(h)}$ for head h are computed as:

$$\mathbf{A}_{ij}^{(h)} = \frac{\mathbf{q}_i^{(h)} \cdot \mathbf{k}_j^{(h)}}{\sqrt{d_k}} + \mathcal{B}_{ij}^{(h)}$$

Where $\mathcal{B}_{ij}^{(h)}$ is the **NL-OOB Bias Term**:

$$\mathcal{B}_{ij}^{(h)} = -\lambda_h \cdot \phi(D_{ij})$$

Here, $\phi(\cdot)$ is the non-linear decay function, and λ_h is the head-specific geometric scaling factor.

4.1.1 The Generalized Metric Tensor ($\mathbf{D}_{ij}$)

Unlike ALiBi, which implicitly computes $\mathbf{D}_{ij} = |i-j|$, NL-OOB requires an explicit distance metric suitable for the data topology. The specification supports lazy evaluation of these metrics to save memory.

Metric Type	Mathematical Definition	Data Domain	Implementation Note
Euclidean	$\ \mathbf{r}_i - \mathbf{r}_j\ $	Molecules, Point Clouds	Compute on-the-fly from coords
Geodesic	$\sum_{e \in \text{path}}$	Knowledge Graphs,	Pre-compute or

	$\text{path} w_e$	Meshes	approx. via landmarks
Hierarchical	$\text{dist}_{\text{tree}}(u, v)$	Code (AST), XML, DOM	Use LCA (Lowest Common Ancestor)
Semantic	$1 - \cos(e_i, e_j)$	RAG, Memory Systems	Based on pre-computed embeddings

Handling Infinity: For disconnected graph components where $D_{ij} = \infty$, the bias term must be masked to $-\infty$.

4.1.2 The Non-Linear Decay Functions (ϕ)

We specify two primary non-linear functions based on the "physics" of the domain:

Type A: Logarithmic (Power Law) Bias

$$\phi(d) = \ln(1 + d)$$

- **Effect:** Attention $\propto (1+d)^{-\lambda_h}$.
- **Use Case:** Scale-free networks, language, molecular potentials, long-range graph interactions.
- **Justification:** Prevents over-squashing; aligns with scale-invariant attention theory.¹⁴

Type B: Quadratic (Gaussian) Bias

$$\phi(d) = d^2$$

- **Effect:** Attention $\propto e^{-\lambda_h d^2}$.
- **Use Case:** Image patches, local surface smoothing, strictly local neighborhoods.
- **Justification:** Approximates the RBF kernel; forces strict locality.

4.1.3 Geometric Head Scaling (λ_h) Specification

We adapt the precise geometric sequence logic from ALiBi¹⁵ to the non-linear domain. In ALiBi, slopes m are powers of 2. For NL-OOB, we generalize this to a **decay ratio** r .

For H heads, we define the scaling factors $\lambda_1, \dots, \lambda_H$:

1. **Calculate Ratio:** $r = (\text{MAX_SCALE})^{\frac{-1}{H}}$ (or similar decay logic).
2. **Generate Sequence:** $\lambda_h = \text{BASE_SCALE} \cdot r^h$.

Detailed Algorithm for Non-Power-of-2 Heads:

Following the robustness requirements found in LabML 15, if the number of heads H is not a power of 2, we must ensure coverage of the entire "scale space."

- Calculate the nearest power of 2, $H_{\text{floor}} = 2^{\lfloor \log_2 H \rfloor}$.
- Generate geometric slopes for H_{floor} .
- For the remaining heads, interpolate strictly between the existing slopes to avoid redundancy and ensure the "attention horizons" are evenly spaced in the log-scale.

Learnable vs. Fixed:

While ALiBi advocates for fixed slopes, NL-OOB recommends Learnable Multipliers. We initialize λ_h using the geometric sequence but allow them to be trainable parameters.

- *Reasoning:* In physics, the exponent p in $1/r^p$ is a specific constant (e.g., 1, 2, 6). By letting the model learn λ_h , it can "discover" the relevant physical forces (e.g., learning $\lambda_h \approx 6$ for Van der Waals).⁴

5. Implementation Strategy and Optimizations

Implementing NL-OOB requires navigating trade-offs between memory usage and computational speed, especially given the quadratic nature of the distance matrix D_{ij} .

5.1 Lazy Computation and Kernel Fusion

A naive implementation computing a $B \times N \times N$ bias tensor will run out of memory (OOM) for sequences longer than ~40% on standard GPUs.

Optimization Strategy:

1. **Lazy Evaluation:** Do not materialize D_{ij} . Pass the coordinate tensors $\mathbf{R} \in \mathbb{R}^{B \times N \times 3}$ directly to the attention function.
2. **Kernel Fusion:** Modify the **FlashAttention** kernel.²¹
 - FlashAttention computes attention in blocks (tiling).
 - Inside the CUDA kernel, when computing the score for block query i and key j , load $\mathbf{r}_i$ and $\mathbf{r}_j$ from SRAM.
 - Compute $d = \|\mathbf{r}_i - \mathbf{r}_j\|$ and bias $s = -\lambda_h \cdot \log(1+d)$ in registers.
 - Add to the dot product before the exp (softmax) step.
 - *Result:* Memory complexity drops from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ (for coordinates).

5.2 Handling "Out of Order" Batches

In standard transformers, masking is usually causal (triangular). In NL-OOB, "Out of Order" implies there is no causality—the mask is determined by connectivity.

- **Graph Masking:** If nodes i and j are not connected (or distance > threshold), we

must inject $-\infty$.

- **Implementation:** Use a sparse distance representation or a block-sparse attention pattern (like BigBird) guided by the geometric locality. High λ heads (local) can use sparse attention; Low λ heads (global) might need global tokens or approximation.

5.3 Code Snippet: PyTorch Implementation

Below is a precise PyTorch implementation of the NL-OOB Bias module, integrating the geometric slope logic from LabML¹⁵ adapted for non-linear power laws.

Python

```
import torch
import torch.nn as nn
import math

class NonLinearOOBBias(nn.Module):
    def __init__(self, num_heads, bias_type='log', max_scale=8.0):
        super().__init__()
        self.num_heads = num_heads
        self.bias_type = bias_type

        # --- Geometric Slope Initialization (Adapting ALiBi Logic) ---
        # We want slopes that cover a range of interaction strengths.
        # For 'log' bias (power law), typical exponents are 1 to 6 (physics).
        # We generate a geometric sequence decaying from max_scale.

        # Calculate ratio to span from max_scale down to a small value (e.g. 0.5)
        # Sequence: max_scale * r^0, max_scale * r^1, ...
        min_scale = 0.5
        decay_rate = math.pow(min_scale / max_scale, 1.0 / (num_heads - 1))

        slopes = torch.tensor([max_scale * (decay_rate ** i) for i in range(num_heads)])

        # Make slopes learnable to allow "discovery" of physical constants
        self.slopes = nn.Parameter(slopes.view(1, num_heads, 1, 1))

    def forward(self, coords_or_dist):
        .....
```

```

Input:
    coords_or_dist: Either (coordinates) or (precomputed dist).
Output:
    bias:
    """
    # 1. Compute Distance D_ij (Lazy if possible)
    if coords_or_dist.dim() == 3 and coords_or_dist.shape[-1] <= 4:
        # Assume coordinates input -> Compute Euclidean Dist
        # Uses efficient broadcasting: (x-y)^2 = x^2 + y^2 - 2xy
        x = coords_or_dist
        x_square = x.pow(2).sum(dim=-1, keepdim=True) #
        # dist^2 = x^2 + y^2 - 2*x@y.T
        dist_sq = x_square + x_square.transpose(1, 2) - 2 * (x @ x.transpose(1, 2))
        # Clamp to 0 for numerical stability
        d_ij = torch.sqrt(torch.clamp(dist_sq, min=1e-6))
    else:
        d_ij = coords_or_dist # Assume precomputed distance matrix

    # 2. Apply Non-Linear Function phi(d)
    if self.bias_type == 'log':
        # Power Law: -lambda * log(1 + d)
        # Corresponds to multiplicative (1+d)^-lambda
        phi_d = torch.log1p(d_ij)

    elif self.bias_type == 'gaussian':
        # Gaussian: -lambda * d^2
        phi_d = d_ij.pow(2)

    # 3. Apply Geometric Slopes
    # Slopes [1, H, 1, 1] * Phi ->
    # Using negative sign to make it a penalty
    bias = -self.slopes * phi_d.unsqueeze(1)

    return bias

```

6. Use Case 1: Molecular Geometry Transformer (MGT)

This use case demonstrates how NL-OOB solves specific failure modes of GNNs in computational chemistry.

6.1 The Problem: Oversquashing in GNNs

Graph Neural Networks rely on message passing. To pass information from atom A to atom B (10 hops away), the signal must pass through 9 intermediate nodes. At each step, the signal is compressed and mixed with neighbors. This leads to **oversquashing**, where long-range interactions (crucial for protein folding or allosteric effects) are lost.¹⁹

Standard Transformers avoid this by being fully connected, but without structural bias, they fail to understand the 3D geometry efficiently, requiring massive datasets to learn what "distance" means.

6.2 The Solution: NL-OOB with Power Law Heads

We design an MGT where the attention mechanism is purely driven by NL-OOB.

- **Input:** Atomic embeddings (Element type, Hybridization).
- **Structure:** 3D coordinates from RDKit conformer generation.
- **Bias Type:** Logarithmic (Power Law).

Head Configuration (Physical Specialization):

We configure 8 heads with learnable slopes initialized geometrically. Based on the analysis of learned power laws 3, we expect the heads to converge to:

- **Head 1 ($\lambda \approx 6.0$): Van der Waals Head.** Extremely local. Attention $\propto r^{-6}$. This head learns to detect steric clashes and packing.
- **Head 2 ($\lambda \approx 1.0$): Coulombic Head.** Attention $\propto r^{-1}$. Captures electrostatic potentials which decay slowly.
- **Head 3 ($\lambda \approx 2.0$): Dipole Head.** Attention $\propto r^{-2}$. Captures dipole-dipole interactions.
- **Heads 4-8 ($\lambda < 0.5$): Global Structural Heads.** These heads "see" the entire molecule, aggregating global descriptors like radius of gyration or overall polarity.

6.3 Performance Expectation

Unlike a GNN, the "Coulombic Head" can directly attend from a positively charged nitrogen to a negatively charged oxygen 15 Angstroms away in a single layer, weighted exactly by the physical $1/r$ law. This provides an **inductive bias** that matches the underlying physics of the domain, leading to faster training (sample efficiency) and better extrapolation to larger molecules (e.g., training on small drugs, testing on peptides).

7. Use Case 2: Hierarchical Code Analysis (AST-Transformer)

Source code is often treated as a sequence of text, but it is fundamentally a tree (Abstract Syntax Tree). Linear ALiBi fails here because two tokens might be adjacent in the text (e.g., }

and function) but far apart in the logical structure.

7.1 The Problem: Linear Bias on Trees

In the code `if (x) { y; } else { z; }`, the token `z` is sequentially far from `if`, but structurally they are siblings in the AST. A linear bias $|i-j|$ penalizes the connection if $i \rightarrow z$ heavily, obscuring the control flow dependency.

7.2 The Solution: NL-OOB with Tree Distance

- **Metric:** Tree Distance $D_{ij} = \text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2 \cdot \text{depth}(\text{LCA}(u, v))$.
- **Bias Type:** Exponential (Linear in log-space) or Gaussian.
- **Head Scaling:**
 - **High Slope:** Attends to parent/children (1-hop).
 - **Medium Slope:** Attends to variable scopes (block level).
 - **Low Slope:** Attends to class/module level dependencies.

By using NL-OOB with AST distances, the model becomes **syntax-aware** by design. It can extrapolate to files with thousands of lines because the *tree depth* rarely grows as fast as the *sequence length*. A variable usage 5000 lines down might only be depth 3 in the tree (Module -> Class -> Method -> Variable), allowing the "Medium Slope" head to attend to its definition effortlessly.

8. Comparative Analysis: NL-OOB vs. State-of-the-Art

We summarize the advantages of NL-OOB through a comparative analysis with leading positional encoding methods.

Feature	Absolute PE / RoPE	Standard ALiBi	Graphormer	NL-OOB (Proposed)
Input Signal	Sequence Index i	Index Diff i	$i-j$	i
Bias Function	Rotation (Sine/Cos)	Linear Penalty	Learnable Scalars	Parametric Non-Linear
Permutation Equivariance	No	No	Yes	Yes

Extrapolation	Poor	Good (1D)	Moderate	Excellent (N-D)
Theoretical Basis	Signal Processing	Recency Heuristic	Graph Theory	Physics & Kernels
Memory Efficiency	High	High	Low (Scalars per hop)	High (Lazy Eval)

Comparison with Graphormer:
Graphormer 23 adds a learnable scalar $b_{\phi(v_i, v_j)}$ for each shortest path distance. This requires learning a unique parameter for distance 1, 2,... K . If the model encounters a distance $K+1$ during inference, it fails (cannot extrapolate).
NL-OOB uses a continuous function $-\lambda \log(1+d)$. It can handle any distance d , even continuous floating-point distances in 3D space, without learning new parameters. This makes NL-OOB strictly superior for continuous extrapolation and metric spaces.¹⁷

9. Conclusion

The specification for **Attention with Non-Linear Out-of-Order Bias (NL-OOB)** represents a necessary evolution of the Transformer architecture for non-sequential domains. By deconstructing ALiBi, we identified that its success lies not in its linearity, but in its **bias-driven geometry**—the use of static, scalable penalties to enforce inductive bias.

NL-OOB generalizes this by:

1. **Decoupling** bias from sequence indices to support "Out-of-Order" sets and graphs.
2. **Replacing** the linear penalty with Power Law or Gaussian functions to prevent over-squashing and match physical/biological realities.
3. **Preserving** the Geometric Head Scaling to create multi-resolution topological processing.

This architecture offers a bridge between the scalability of Transformers and the structural awareness of GNNs. For practitioners in scientific computing, code analysis, and knowledge representation, NL-OOB provides a robust, physically grounded, and computationally efficient mechanism to model the complex geometries of the real world. Future work should focus on hardware-accelerated kernels for on-the-fly metric computation, which will unlock the full potential of this structural foundation model.

Data Clusters & Citations:

- ALiBi Design & Code: ¹
- Length Extrapolation: ²
- Physical Power Laws & Biases: ³
- Neuroscience/Visual Cortex: ⁵
- Kernel Theory/Kerple: ¹⁶
- Graphormer Comparison: ²³
- FlashAttention Optimization: ²¹
- Scale-Invariant Attention: ¹⁴

Works cited

1. ALiBi: Attention with Linear Biases | ML & CV Consultant - Abhik Sarkar, accessed December 2, 2025, <https://www.abhik.xyz/concepts/attention/alibi>
2. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation, accessed December 2, 2025, <https://arxiv.org/abs/2108.12409>
3. (PDF) Power law attention biases for molecular transformers - ResearchGate, accessed December 2, 2025, https://www.researchgate.net/publication/397663750_Power_law_attention_biases_for_molecular_transformers
4. Power law attention biases for molecular transformers - arXiv, accessed December 2, 2025, <https://arxiv.org/pdf/2511.11489>
5. A universal power law optimizes energy and representation fidelity in visual adaptation - NIH, accessed December 2, 2025, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11974702/>
6. Power law attention biases for molecular transformers - arXiv, accessed December 2, 2025, <https://arxiv.org/html/2511.11489v1>
7. Transformer (deep learning) - Wikipedia, accessed December 2, 2025, [https://en.wikipedia.org/wiki/Transformer_\(deep_learning\)](https://en.wikipedia.org/wiki/Transformer_(deep_learning))
8. Positional Encoding Explained: A Deep Dive into Transformer PE - Medium, accessed December 2, 2025, <https://medium.com/thedeephub/positional-encoding-explained-a-deep-dive-into-transformer-pe-65cfe8cfe10b>
9. The Impact of Positional Encoding on Length Generalization in Transformers - arXiv, accessed December 2, 2025, <https://arxiv.org/pdf/2305.19466>
10. The Use Case for Relative Position Embeddings - Ofir Press, accessed December 2, 2025, <https://ofir.io/The-Use-Case-for-Relative-Position-Embeddings/>
11. Relative vs. Absolute Positional Embedding in Decoders - Newline.co, accessed December 2, 2025, <https://www.newline.co/@zaoyang/relative-vs-absolute-positional-embedding-in-decoders--bcbe7df9>
12. Beyond Attention: How Advanced Positional Embedding Methods Improve upon the Original Approach in Transformer Architecture | by Elahe Aghapour - Medium,

- accessed December 2, 2025,
<https://medium.com/data-science/beyond-attention-how-advanced-positional-embedding-methods-improve-upon-the-original-transformers-90380b74d324>
13. ALiBi: Attention with Linear Biases - ML Digest, accessed December 2, 2025,
<https://ml-digest.com/alibi-attention-with-linear-biases/>
 14. (PDF) Scale-invariant Attention - ResearchGate, accessed December 2, 2025,
https://www.researchgate.net/publication/392086040_Scale-invariant_Attention
 15. Attention with Linear Biases (ALiBi) - labml.ai, accessed December 2, 2025,
<https://nn.labml.ai/transformers/alibi/index.html>
 16. (PDF) KERPLE: Kernelized Relative Positional Embedding for Length Extrapolation, accessed December 2, 2025,
https://www.researchgate.net/publication/360782542_KERPLE_Kernelized_Relative_Positional_Embedding_for_Length_Extrapolation
 17. KERPLE: Kernelized Relative Positional Embedding for Length Extrapolation - arXiv, accessed December 2, 2025, <https://arxiv.org/pdf/2205.09921>
 18. Gaussian Blur - Standard Deviation, Radius and Kernel Size, accessed December 2, 2025,
<https://dsp.stackexchange.com/questions/10057/gaussian-blur-standard-deviation-radius-and-kernel-size>
 19. Long-Context Generalization with Sparse Attention - arXiv, accessed December 2, 2025, <https://arxiv.org/html/2506.16640v3>
 20. On the Emergence of Position Bias in Transformers - arXiv, accessed December 2, 2025, <https://arxiv.org/pdf/2502.01951?>
 21. Relative positions, or Alibi, or attention_bias (T5) support · Issue #332 · Dao-AI Lab/flash-attention - GitHub, accessed December 2, 2025,
<https://github.com/Dao-AI-Lab/flash-attention/issues/332>
 22. ALiBi FlashAttention - Speeding up ALiBi by 3-5x with a hardware-efficient implementation, accessed December 2, 2025,
<https://pli.princeton.edu/blog/2024/alibi-flashattention-speeding-alibi-3-5x-hardware-efficient-implementation>
 23. Graphormer Boosted: Molecular Property Prediction With Enhanced Graph Spatial and Edge Encodings - IEEE Xplore, accessed December 2, 2025,
<https://ieeexplore.ieee.org/iel8/6287639/10820123/11091319.pdf>
 24. KERPLE: Kernelized Relative Positional Embedding for Length Extrapolation, accessed December 2, 2025,
https://papers.neurips.cc/paper_files/paper/2022/file/37a413841a614b5414b333585e7613b8-Paper-Conference.pdf
 25. Power Laws: How Nonlinear Relationships Amplify Results - Farnam Street, accessed December 2, 2025, <https://fs.blog/power-laws/>
 26. Build Model — DGL 2.5 documentation, accessed December 2, 2025,
https://www.dgl.ai/dgl_docs/graphtransformer/model.html
 27. A Survey of Graph Transformers: Architectures, Theories and Applications - arXiv, accessed December 2, 2025, <https://arxiv.org/pdf/2502.16533>
 28. Log-Linear Attention - arXiv, accessed December 2, 2025,
<https://arxiv.org/pdf/2506.04761>